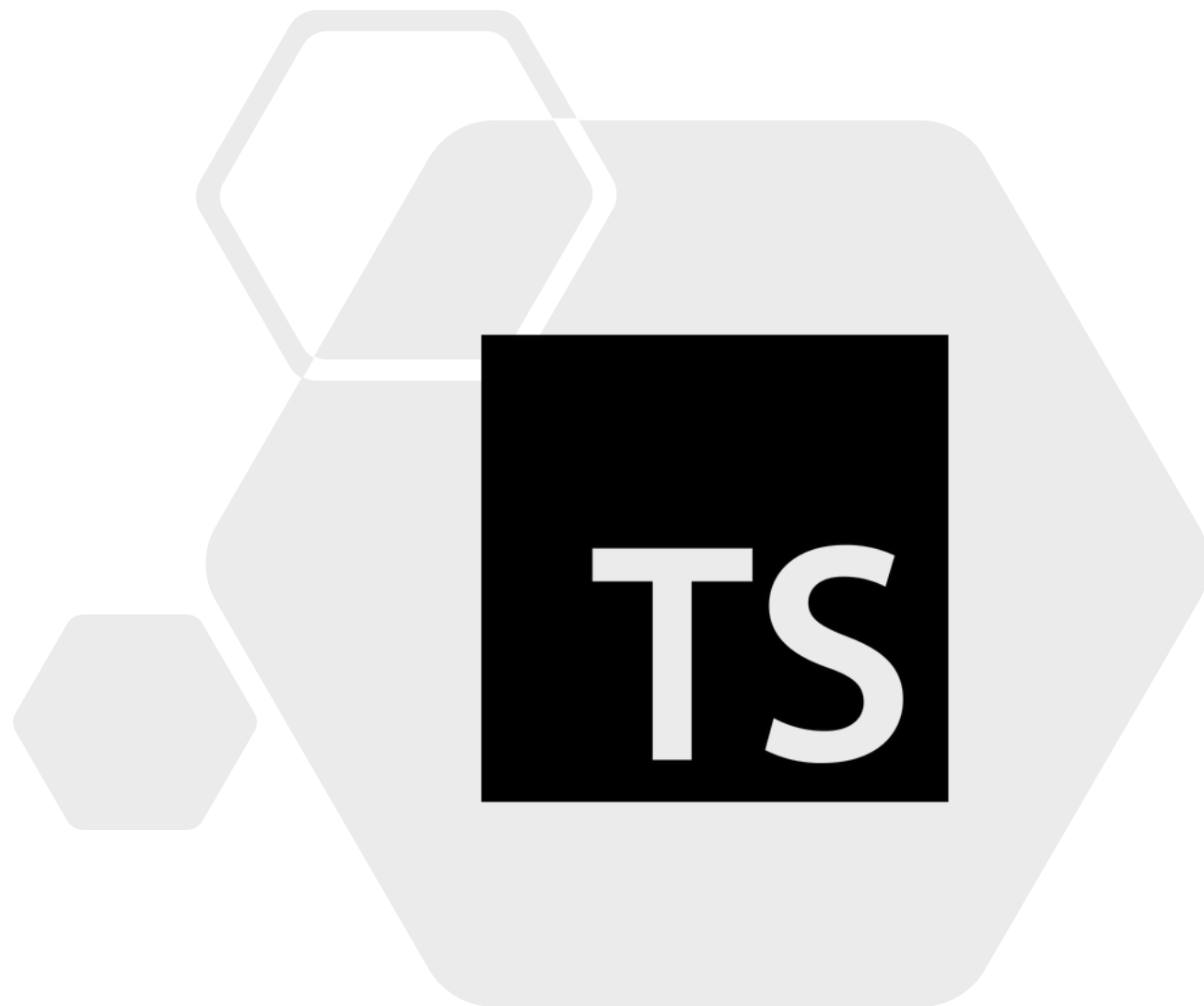
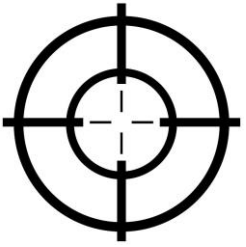


Padrões
criacionais



Padrões criacionais



Os padrões de design criacionais são técnicas que sugerem mecanismos de criação de objetos. O objetivo é criar objetos de uma maneira adequada à situação, dado que a forma básica de criação de objetos pode resultar em problemas de design ou em complexidade adicional ao design do projeto.

Os padrões criacionais visam separar um sistema de como seus objetos são criados, compostos e representados. Eles aumentam a flexibilidade do sistema em termos de o quê, quem, como e quando da criação do objeto.

Singleton



O singleton é um padrão de design que restringe a instanciação de uma classe a uma instância “única”, um único objeto.

Geralmente, utiliza-se o singleton para controlar o acesso a um recurso compartilhado. Por exemplo, acesso a base dados, acesso a algum arquivo, acesso a algum serviço etc.

Implementação singleton

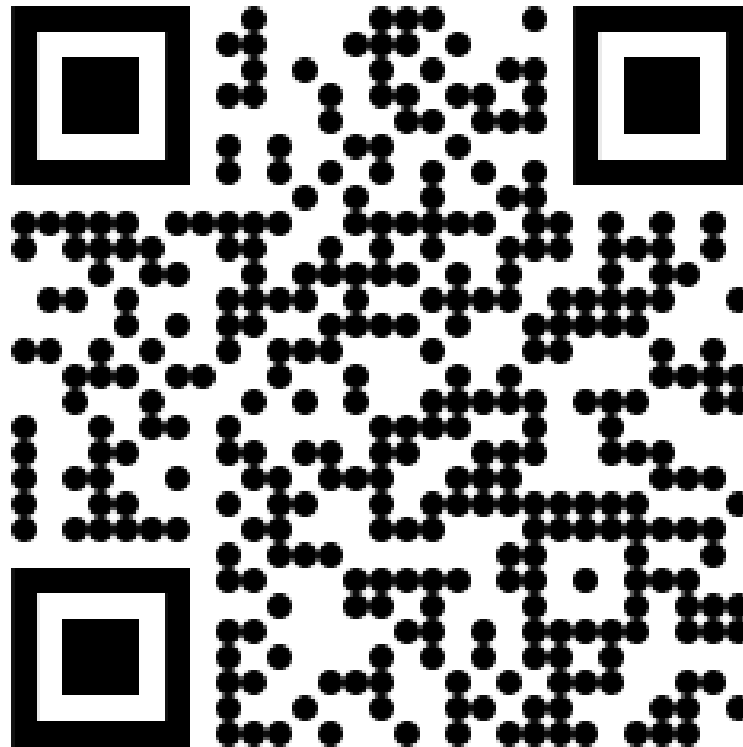
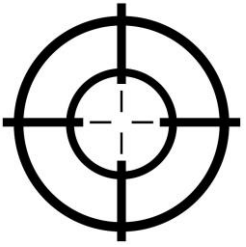


```
class Classe {  
    private static instanciaUnica: Classe;  
    private constructor() { }  
    public static obterInstancia(): Classe {  
        if (!Classe.instanciaUnica) {  
            this.instanciaUnica = new Classe();  
        }  
        return this.instanciaUnica;  
    }  
}
```

```
class Classe {  
    private static instanciaUnica: Classe = new Classe();  
    private constructor() {}  
    public static obterInstancia(): Classe {  
        return this.instanciaUnica;  
    }  
}
```

A implementação do singleton segue dois passos comuns. Fazer o construtor padrão privado para evitar a criação padrão e criar um objeto estático da própria classe, que armazena a instancia única.

Implementação singleton



<https://github.com/gerson-pn/singleton-typeScript>

Singleton é um anti-padrão



Sim! O singleton é considerado, por muitos, um anti-padrão. Isto porque ele fere o princípio da responsabilidade única, que é um dos princípios do SOLID.

Isto acontece porque a classe já é responsável pela criação de sua instância única. Então, se mais alguma lógica for adicionada na classe ela passa a ter mais de uma responsabilidade.

Mesmo assim ele pode ser útil! Utilizar ou não é critério do desenvolvedor e suas justificativas.

Prototype

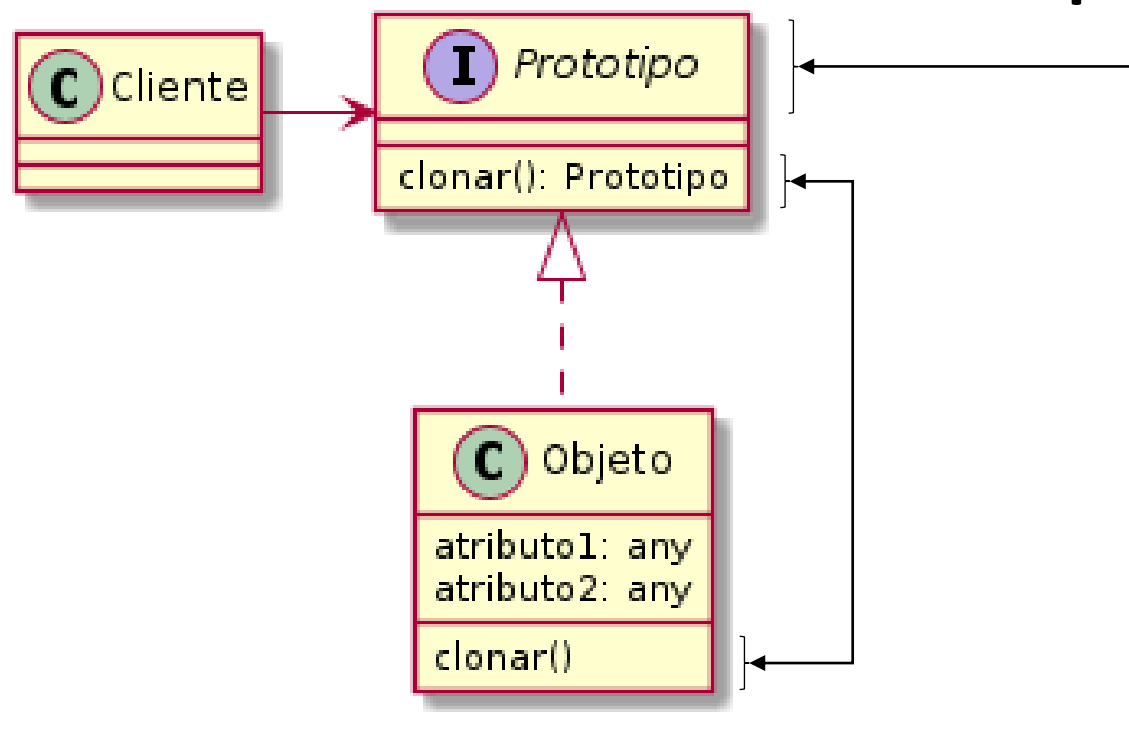
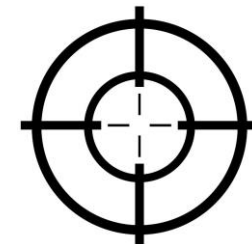


O protótipo é usado para copiar objetos existentes, sem o custo de conhecer ou depender de suas classes.

Copiar objetos geralmente é uma tarefa simples, mas há exceções, que ocorrem quando há campos privados na classe ou quando se conhece apenas a interface de uma hierarquia e não suas classes concretas.

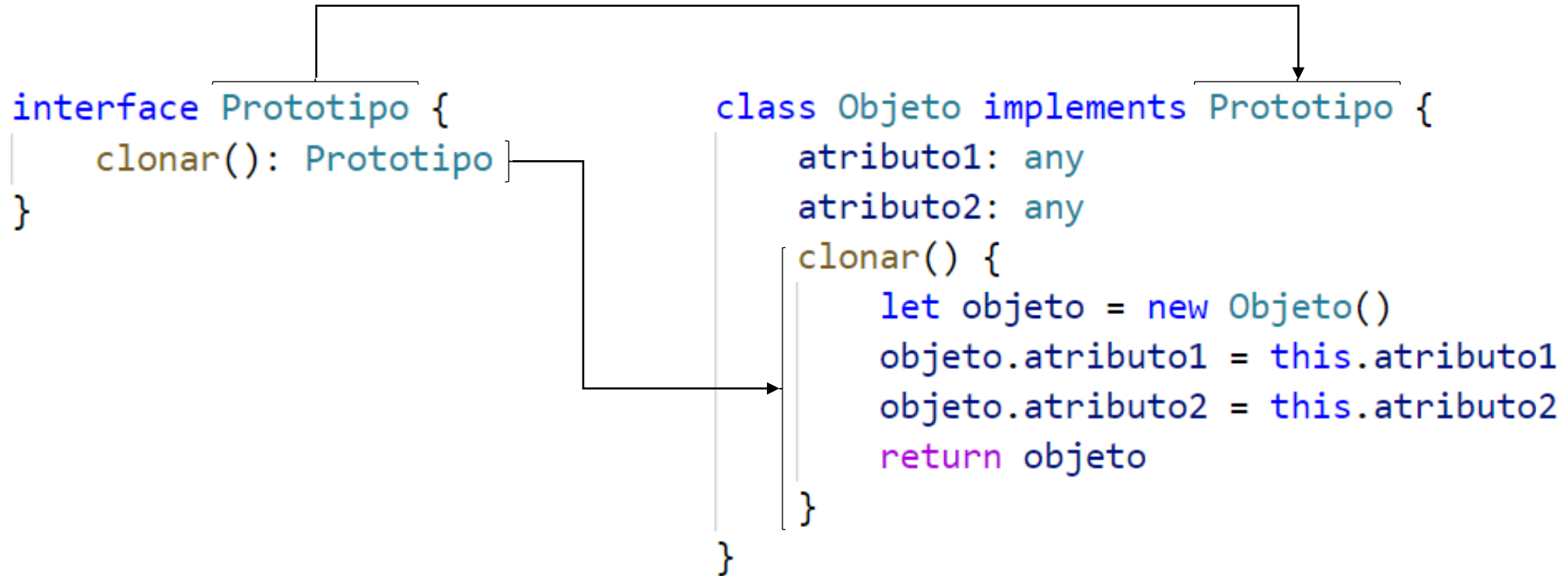
Outro problema ocorre quando o código, que faz a cópia, é dependente do tipo da classe a ser copiada.

Prototype UML



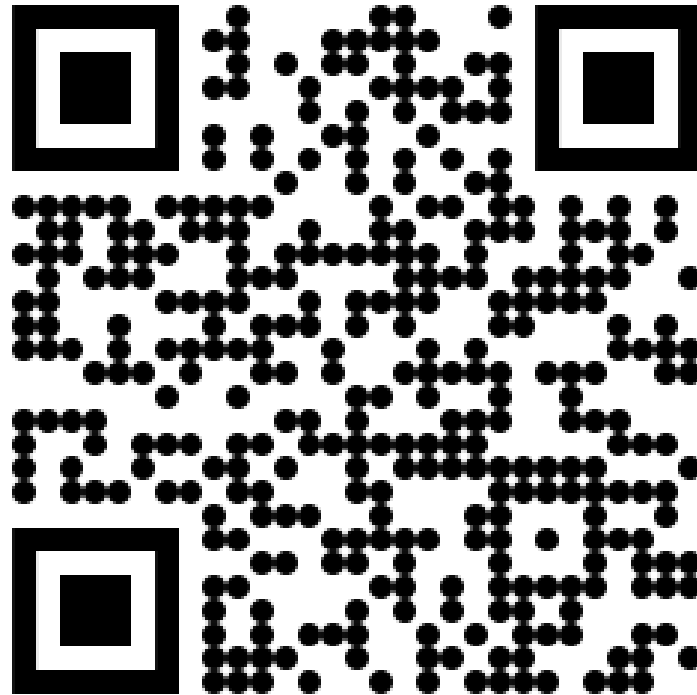
A implementação do protótipo, geralmente, utiliza uma interface. Esta interface contém um método genérico para clonagem e deve ser implementada na classe passível de clonagem. O padrão delega a responsabilidade da clonagem para o próprio objeto.

Implementação Prototype



A classe concreta torna-se responsável por implementar o método de clonagem.

Implementação Prototype



<https://github.com/gerson-pn/prototype-typescript>

Vantagens do prototype



O padrão protótipo entrega mais vantagens do que uma forma de copiar objetos. Outra vantagem de utilizá-lo é que ele pode auxiliar na criação de objetos repetidos ou com algum procedimento de inicialização base, com em objetos pré-construídos.

Utilizar objetos pré-construídos, como base para criar outros, é mais conveniente, principalmente quando se trata de objetos com muitos atributos.

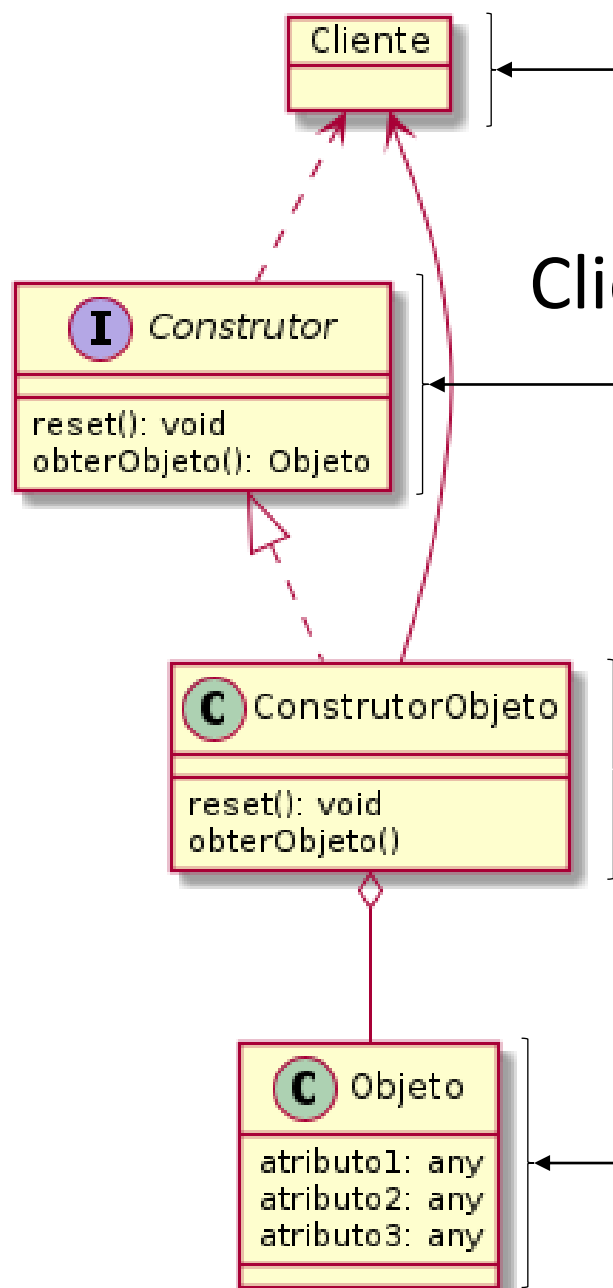
OBS: A linguagem TypeScript fornece uma alternativa já pronta para o padrão.

Builder

O padrão construtor é um padrão de projeto que fornece uma solução flexível criação de objetos. O objetivo deste padrão de projeto é separar a construção de um objeto complexo de sua representação, sua definição em classe.

A solução proposta pelo padrão é extrair o código de construção do objeto para fora de sua própria classe, a responsabilidade da construção é transferida para objetos denominados de “builders” (construtores).

Builder UML



Cliente, parte do sistema consumidora de objetos.

Interface, com as assinaturas de método que todo construtor deve implementar.

Implementação concreta, classe responsável pelos métodos de inserção dos atributos.

Classe que define o modelo de objeto a ser criado, pelo construtor.

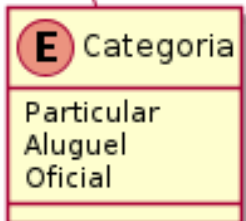
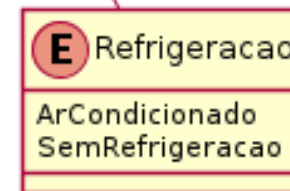
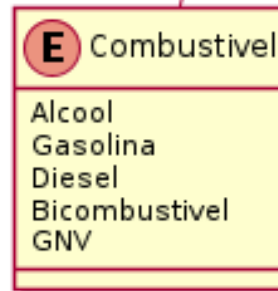
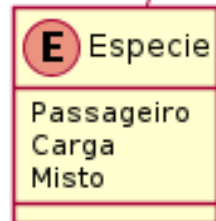
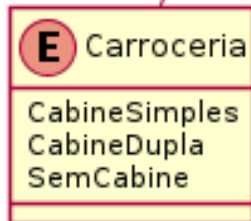
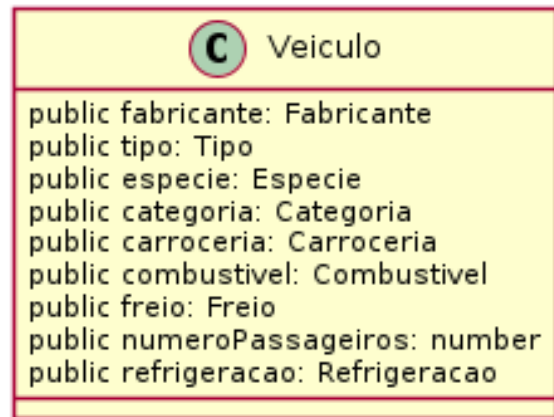
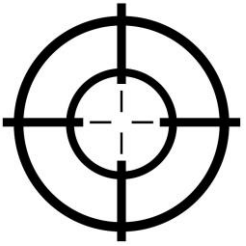
```
export default interface Construtor {
  reset(): void
  obterObjeto(): Objeto
}
```

```
export default class Objeto {
  atributo1: any
  atributo2: any
  atributo3: any
}
```

```
let construtor = new ConstrutorObjeto()
construtor.setAtributo1("A")
construtor.setAtributo3("C")
let objeto = construtor.obterObjeto()
```

```
export default class ConstrutorObjeto implements Construtor {
  private objeto!: Objeto
  constructor(){
    this.reset()
  }
  public setAtributo1(atributo1: any) {
    this.objeto.atributo1 = atributo1
  }
  public setAtributo2(atributo2: any) {
    this.objeto.atributo2 = atributo2
  }
  public setAtributo3(atributo3: any) {
    this.objeto.atributo3 = atributo3
  }
  reset(): void {
    this.objeto = new Objeto()
  }
  obterObjeto() {
    return this.objeto
  }
}
```

Builder Exemplo



Quantas possibilidades de criar um veículo?

```

export default class ConstrutorVeiculo implements Construtor {
  private veiculo!: Veiculo;
  constructor() { ... }
  public setFabricante(fabricante: Fabricante) { ... }
  ...
  public setNumeroPassageiros(numeroPassageiros: number) { ... }
  public setRefrigeracao(refrigeracao: Refrigeracao) { ... }
  reset(): void {
    this.veiculo = new Veiculo()
  }
  obterObjeto() {
    return this.veiculo
  }
}

let construtor = new ConstrutorVeiculo()
construtor.setCombustivel(Combustivel.Bicombustivel)
construtor.setEspecie(Especie.Passageiro)
construtor.setTipo(Tipo.Automovel)
let veiculo = construtor.obterObjeto()

```

```

class Veiculo {
  public fabricante!: Fabricante
  public tipo!: Tipo
  public especie!: Especie
  public categoria!: Categoria
  public carroceria!: Carroceria
  public combustivel!: Combustivel
  public freio!: Freio
  public numeroPassageiros!: number
  public refrigeracao!: Refrigeracao
}

export default interface Construtor {
  reset(): void
  obterObjeto(): Veiculo
}

```


Implementação Builder



<https://github.com/gerson-pn>

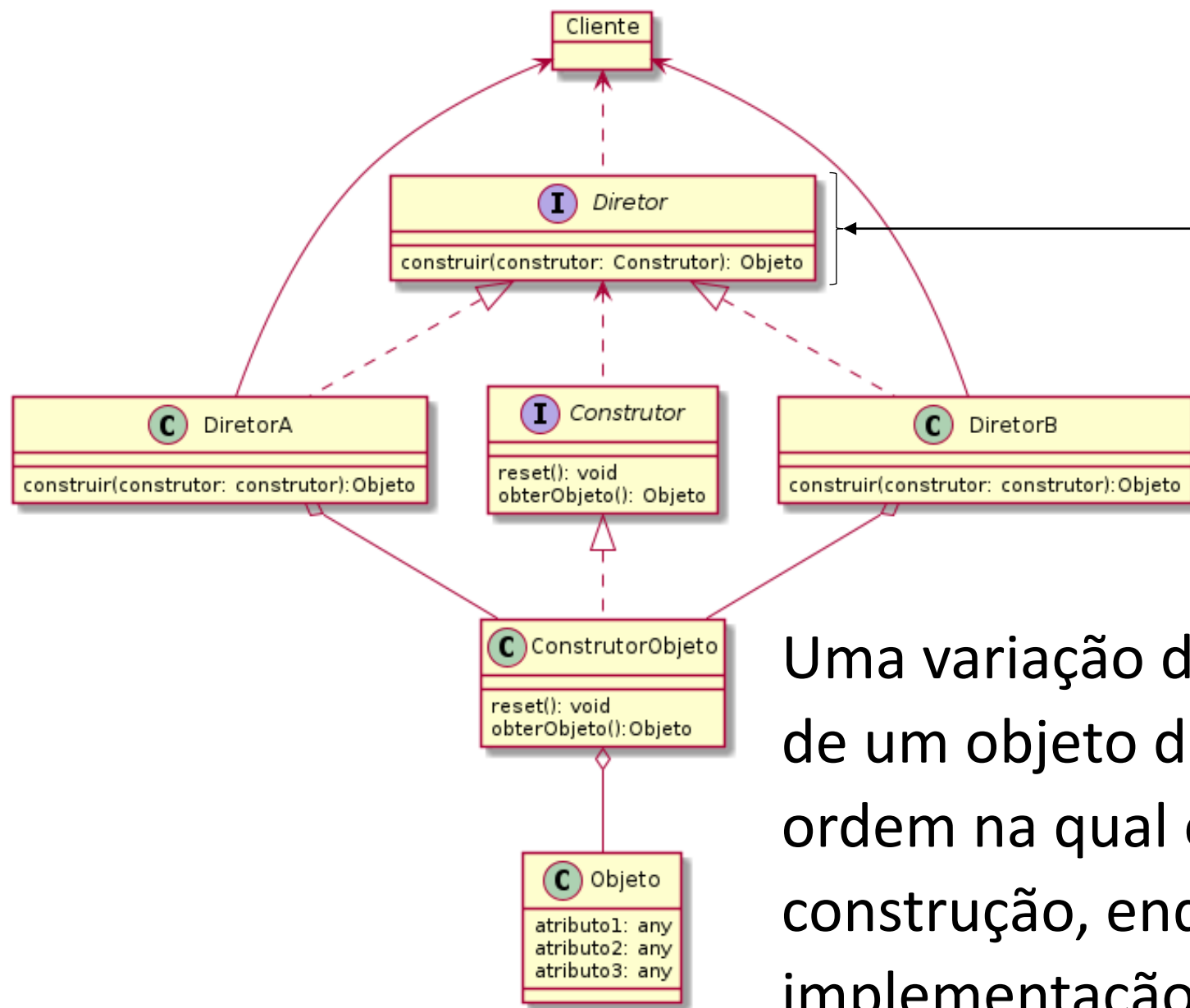
/builder-simple-architecture



/builder-simple-example-typeScript



Builder UML



Uma variação do padrão construtor é a inclusão de um objeto diretor. A classe diretor define a ordem na qual executar as etapas de construção, enquanto o construtor provê a implementação dessas etapas.

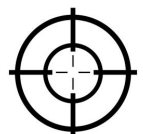
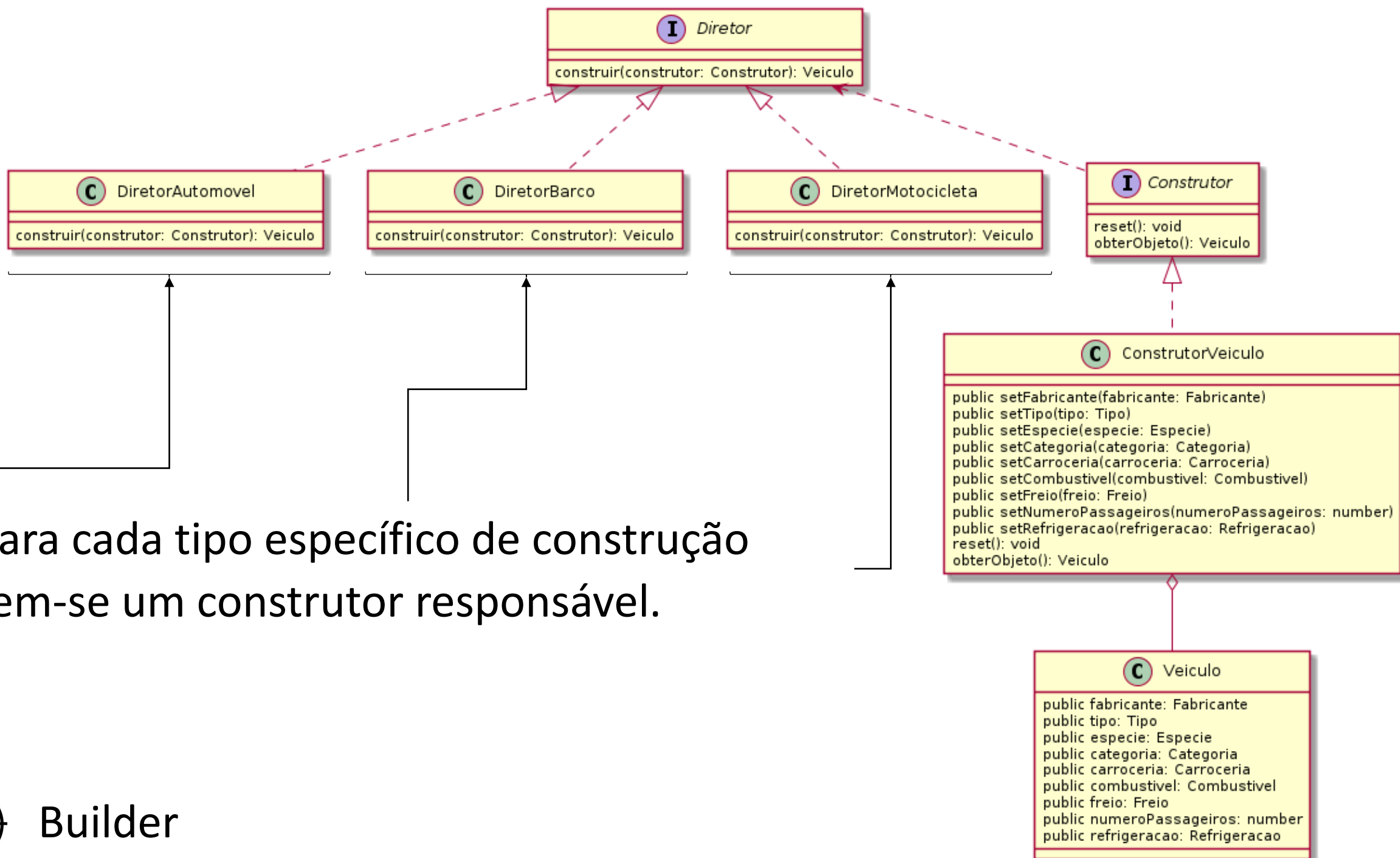
```
export default class DiretorA implements Diretor {
  construir(construtor: construtor) {
    let construtorA = construtor as ConstrutorObjeto
    construtorA.setAtributo1('A')
    construtorA.setAtributo1('A')
    construtorA.setAtributo1('A')
    return construtorA.obterObjeto()
  }
}
```

```
export default class DiretorB implements Diretor {
  construir(construtor: construtor) {
    let construtorB = construtor as ConstrutorObjeto
    construtorB.setAtributo1('B')
    construtorB.setAtributo1('B')
    construtorB.setAtributo1('B')
    return construtorB.obterObjeto()
  }
}
```

```
export default interface Diretor {
  construir(construtor: Construtor): Objeto
}
```

```
export default interface Construtor {
  reset(): void
  obterObjeto(): Objeto
}
```

Cada diretor sabe como “ordenar” a construção de uma variação do objeto.



Builder

```
export default interface Construtor {  
  reset(): void  
  obterObjeto(): Veiculo  
}
```

```
export default interface Diretor {  
  construir(construtor: Construtor): Veiculo  
}
```

```
export default class DiretorAutomovel implements Diretor {  
  construir(construtor: Construtor): Veiculo {  
    let constructorAutomovel = construtor as ConstrutorVeiculo  
    constructorAutomovel.setRefrigeracao(Refrigeracao.SemRefrigeracao)  
    constructorAutomovel.setCategoria(Categoria.Particular)  
    constructorAutomovel.setEspecie(Especie.Passageiro)  
    constructorAutomovel.setCarroceria(Carroceria.CabineSimples)  
    constructorAutomovel.setFreio(Freio.ABS)  
    constructorAutomovel.setTipo(Tipo.Automovel)  
    constructorAutomovel.setNumeroPassageiros(5)  
    constructorAutomovel.setCombustivel(Combustivel.Bicombustivel)  
    return constructorAutomovel.obterObjeto();  
  }  
}
```

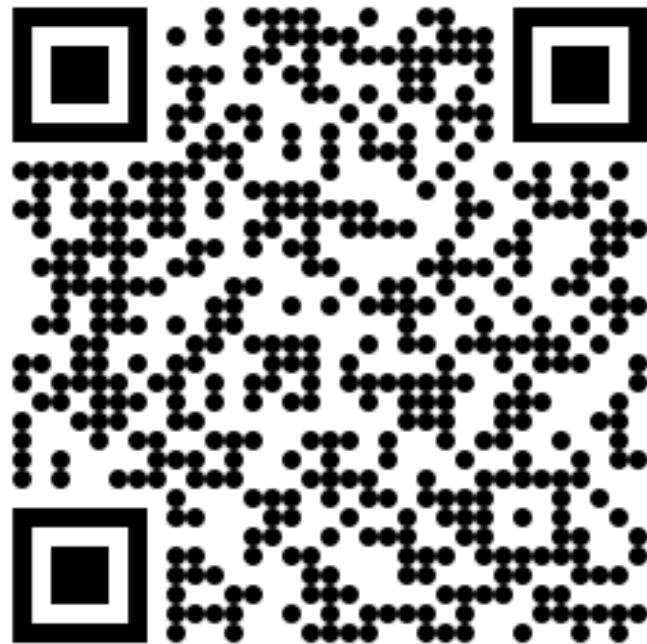
```
export default class DiretorBarco implements Diretor {  
  construir(construtor: Construtor): Veiculo { ...  
  }  
}
```

Implementação Builder



<https://github.com/gerson-pn>

/builder-example-director



/builder-example-director-typeScript



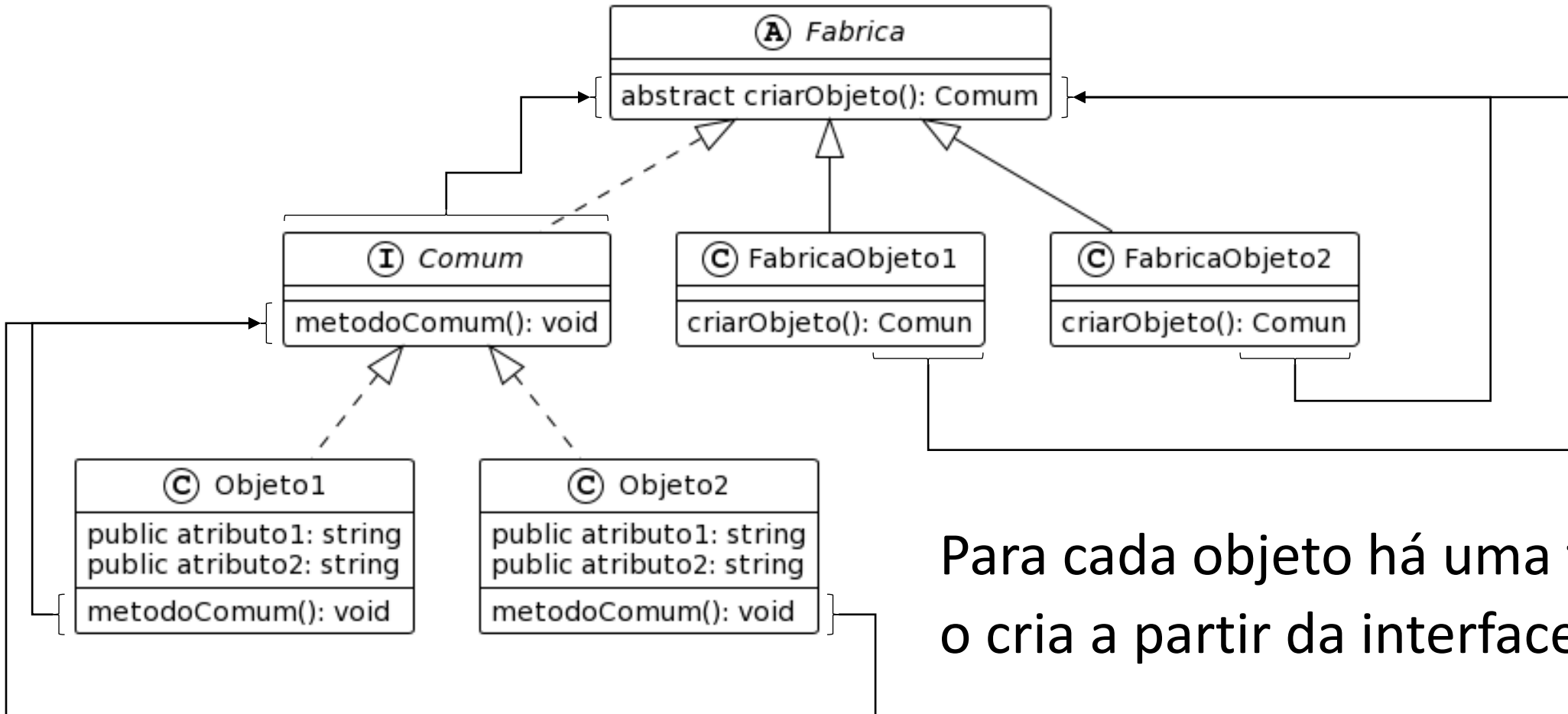
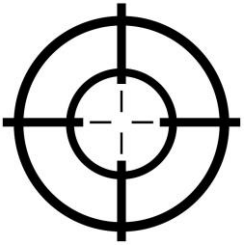
Método fábrica



O método fábrica - factory method, também chamado de construtor virtual, é um padrão criacional utilizado como alternativa ao uso do método construtor.

A proposta deste padrão é utilizar um método virtual (interface) para criar objetos de uma determinada hierarquia, sem chamada direto ao método construtor ou a palavra reservada “new”. Desta forma, o tipo dos objetos criados pode variar mantendo um único método como canal de criação.

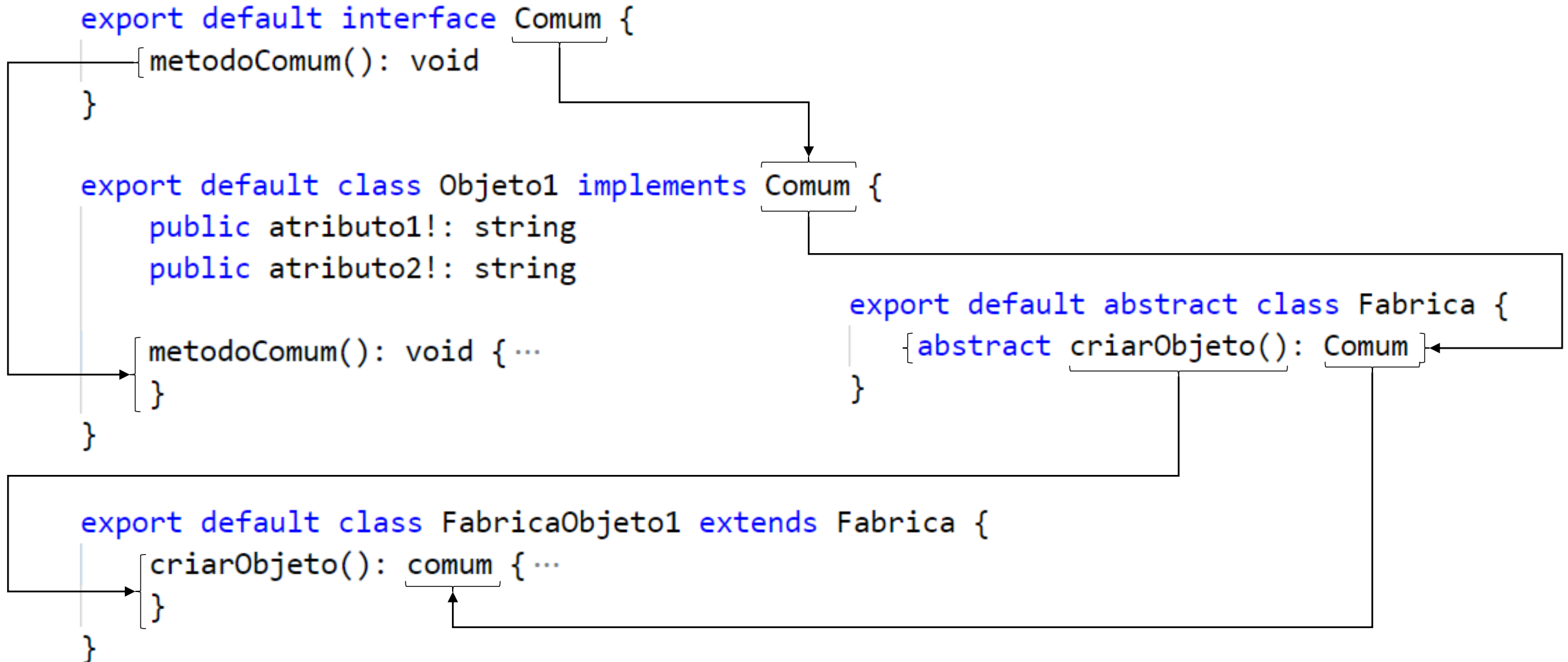
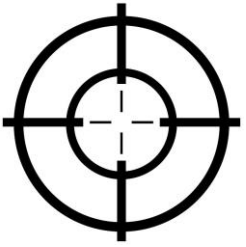
Método fábrica UML

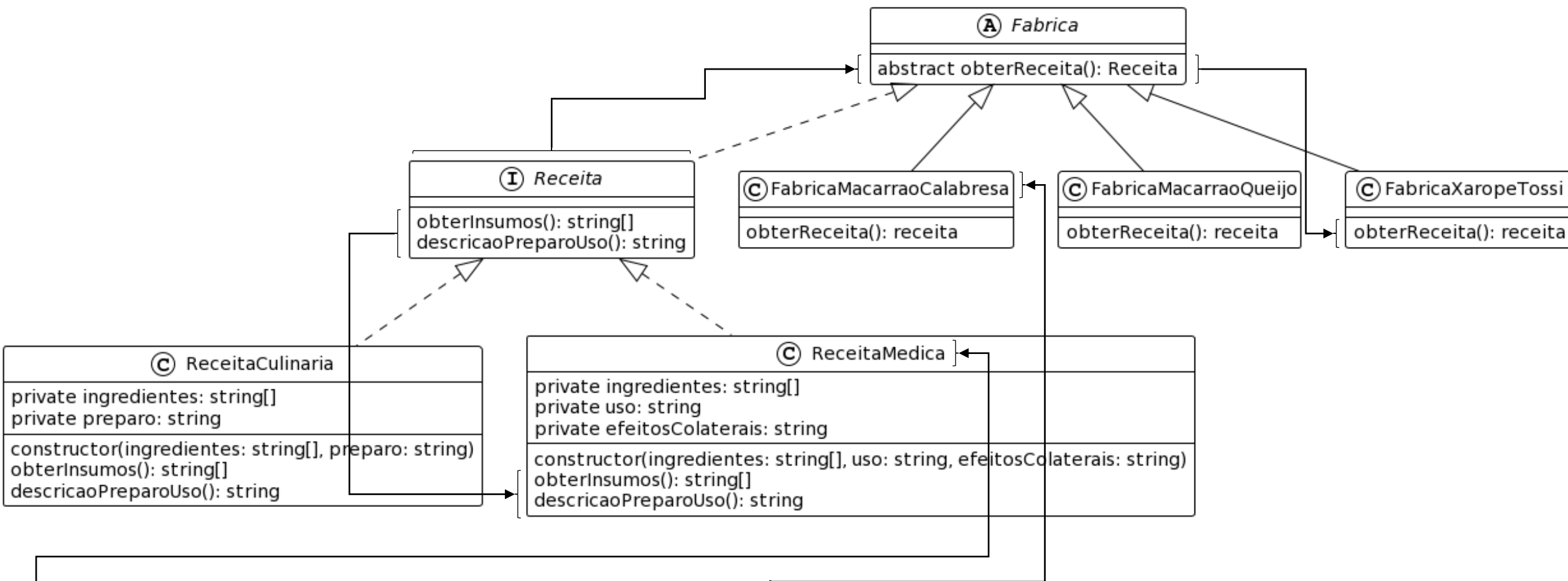


Para cada objeto há uma fábrica, que o cria a partir da interface.

A classes concretas são abstraídas para uma interface em comum, com métodos que sejam comuns para todos.

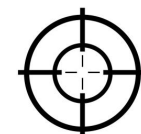
Método fábrica





Neste exemplo existem duas classes concretas, que representam receitas de tipos diferentes. Para cada especificidade de classe há uma fábrica, responsável por criar a receita corretamente.

Método fábrica



```
export default interface Receita {
  obterInsumos(): string[]
  descricaoPreparoUso(): string
}
```

```
export default class FabricaMacarraoCalabresa extends Fabrica {
  obterReceita(): receita { ...
}
```

```
export default class FabricaXaropeTossi extends Fabrica {
  obterReceita(): receita { ...
}
```

```
export default abstract class Fabrica {
  abstract obterReceita(): Receita
}
```

```
export default class ReceitaCulinaria implements Receita {
  private ingredientes: string[]
  private preparo: string
  constructor(ingredientes: string[], preparo: string) { ...
  }
  obterInsumos(): string[] { ...
  }
  descricaoPreparoUso(): string { ...
  }
}
```

Método fábrica



Método fábrica



O método fábrica separa o código de construção dos objetos do código que realmente usa os objetos. Isto facilita o reuso de código e a aplicação de tipos diferentes de fábricas ao longo da aplicação.

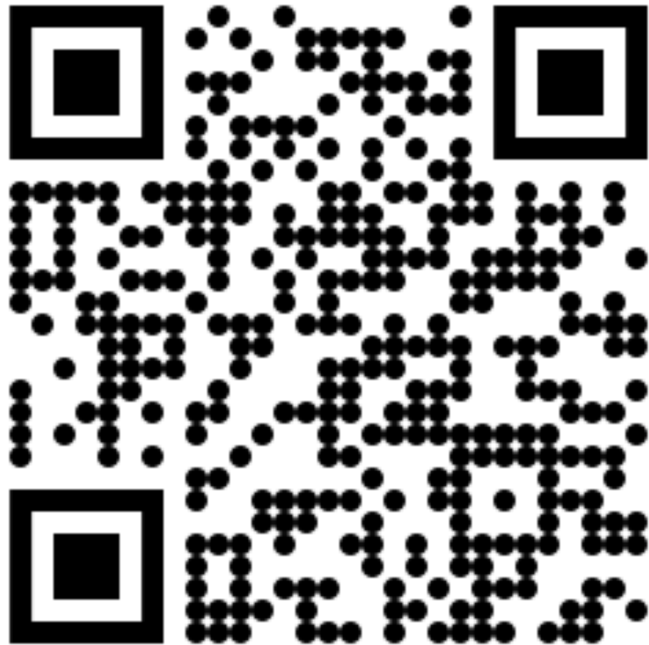
A estrutura do padrão evita acoplamentos de código desnecessários entre o criador e os objetos concretos. Além disso, a estrutura do padrão evidencia o princípio da responsabilidade única e o princípio aberto/fechado.

Método fábrica

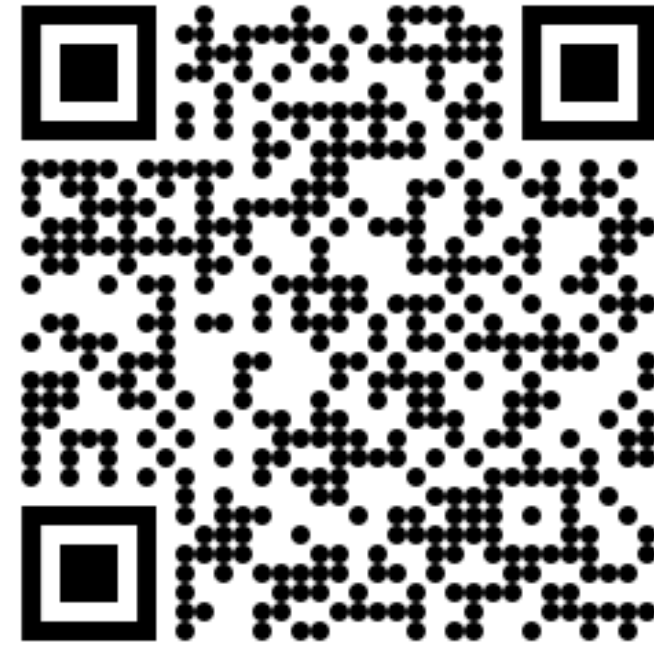


<https://github.com/gerson-pn>

[/factory-method-example](#)



[/factory-method-example-typeScript](#)



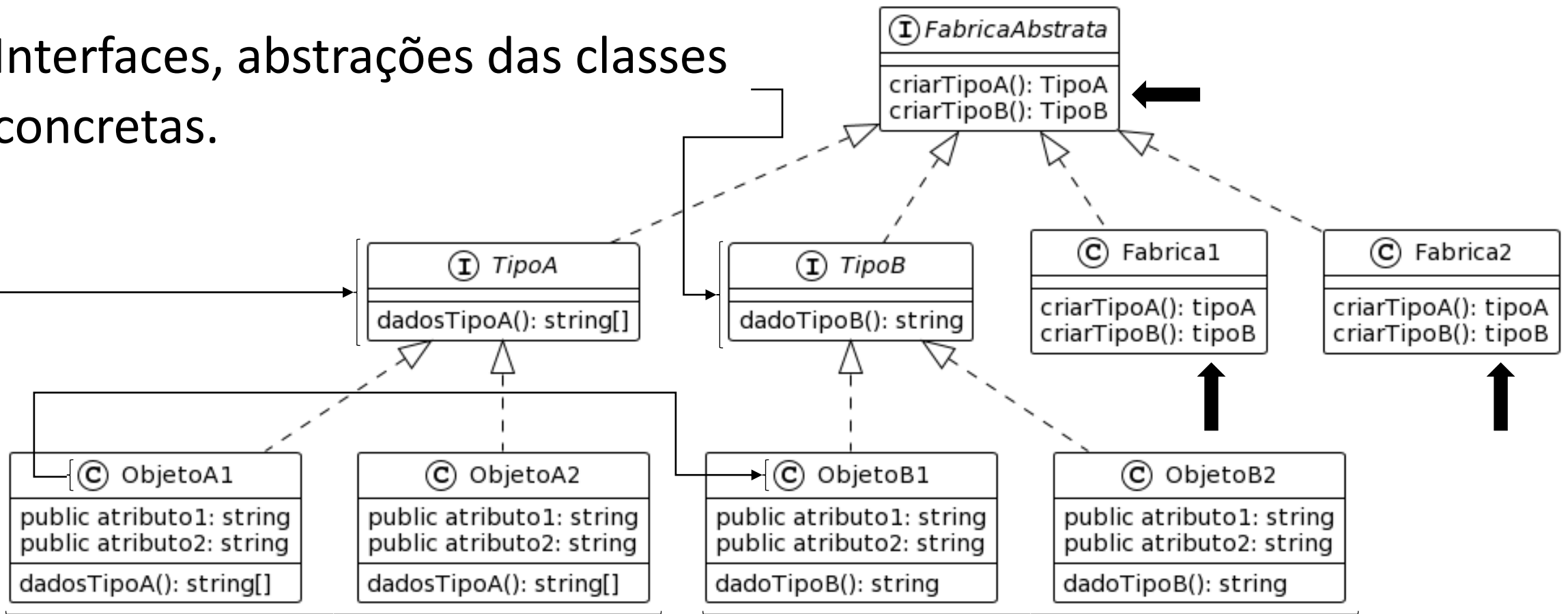
Fábrica abstrata



O padrão de projeto fábrica abstrata (abstract factory) permite a criação de famílias de objetos relacionados ou dependentes por meio de uma única interface, sem especificar suas classes concretas.

Este padrão propõe declarar explicitamente interfaces para cada objeto distinto de uma família de objetos, as variantes desses objetos devem seguir essas interfaces. Com isso, pode-se abstrair a criação desses objetos a partir das interfaces, por um único lugar, uma fábrica genérica.

Interfaces, abstrações das classes concretas.



Classes concretas que pertencem à mesma família ou estão relacionadas.

Fábrica abstrata 

Tipo A.

Tipo B.

Categoria 1.

© ObjetoA1
public atributo1: string public atributo2: string
dadosTipoA(): string[]

© ObjetoB1
public atributo1: string public atributo2: string
dadoTipoB(): string

Categoria 2.

© ObjetoA2
public atributo1: string public atributo2: string
dadosTipoA(): string[]

© ObjetoB2
public atributo1: string public atributo2: string
dadoTipoB(): string

Objetos correlacionados!

Objetos da mesma família ou hierarquia!

Fábrica abstrata 


```
export default interface TipoA {  
  dadosTipoA(): string[]  
}
```

```
export default interface TipoB {  
  dadoTipoB(): string  
}
```

```
export default class ObjetoB2 implements TipoB {  
  public atributo1: string  
  public atributo2: string  
  dadoTipoB(): string { ...  
}
```

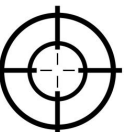
```
export default class ObjetoB1 implements TipoB {  
  public atributo1: string  
  public atributo2: string  
  dadoTipoB(): string { ...  
}
```

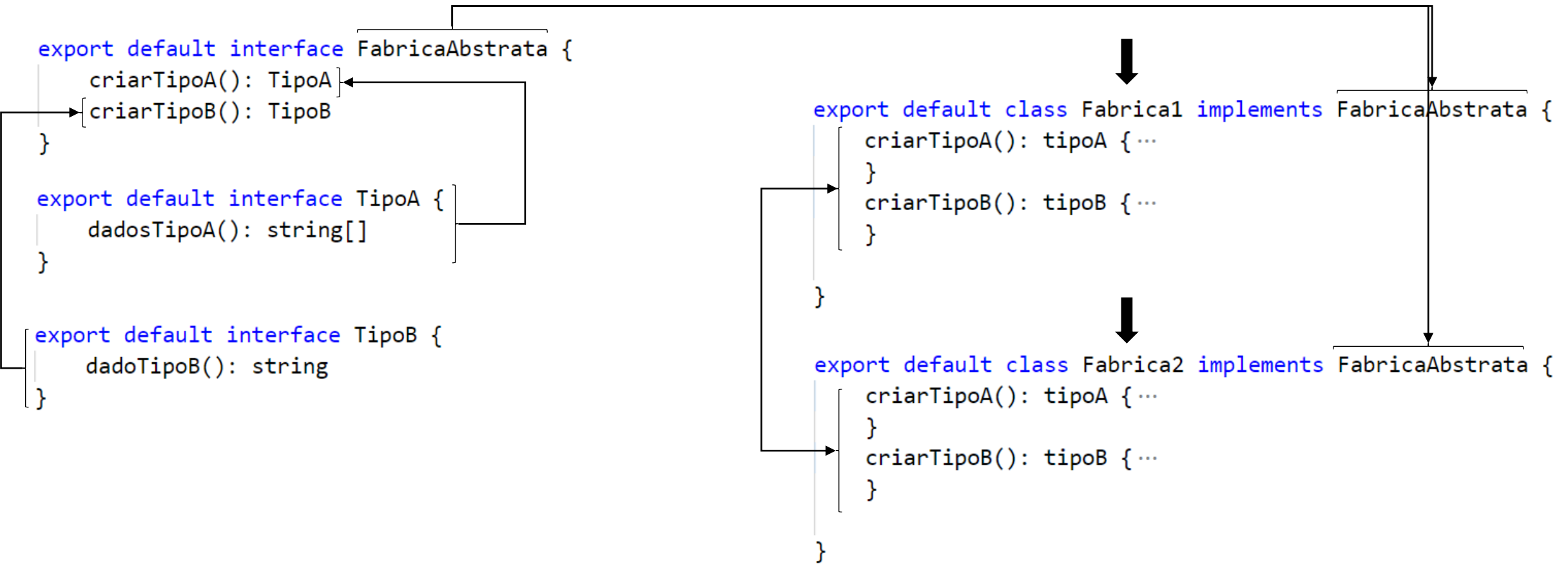
```
export default class ObjetoA1 implements TipoA {  
  public atributo1: string  
  public atributo2: string  
  
  dadosTipoA(): string[] { ...  
}
```

```
export default class ObjetoA2 implements TipoA {  
  public atributo1: string  
  public atributo2: string  
  
  dadosTipoA(): string[] { ...  
}
```

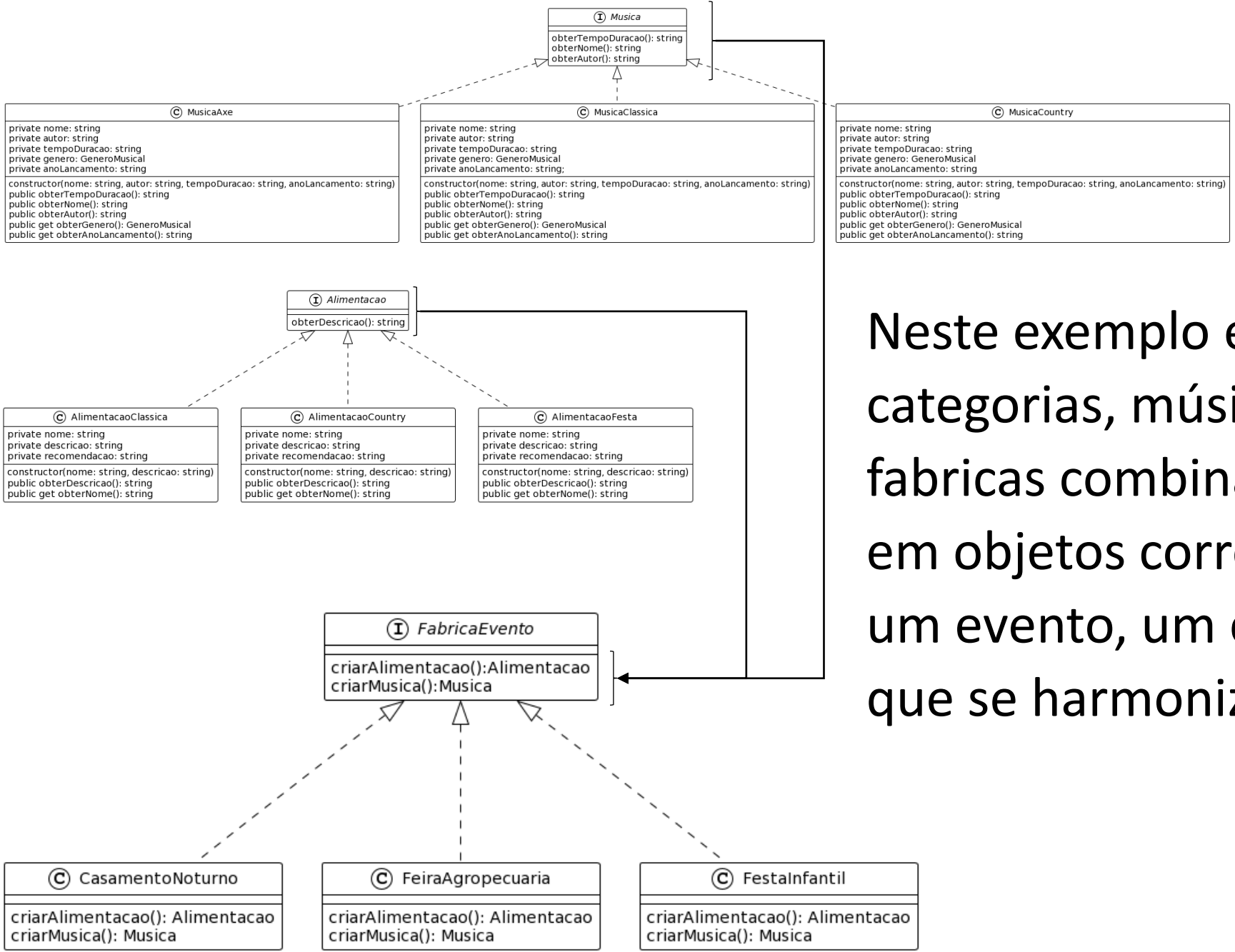
Os objetos são abstraídos para interfaces.

Fábrica abstrata





Cada fábrica contém uma forma específica de combinar os objetos correlacionados e suas famílias.



Neste exemplo existem duas categorias, música e alimentação. As fabricas combinam estas categorias em objetos correlacionados para criar um evento, um conjunto de objetos que se harmonizam.

As fábricas criam eventos específicos a partir de abstrações (interfaces)!

```
export default interface FabricaEvento{  
  criarAlimentacao():Alimentacao  
  criarMusica():Musica  
}
```

```
export default class FeiraAgropecuaria implements FabricaEvento {  
  criarAlimentacao(): alimentacao { ...  
  }  
  criarMusica(): musica { ...  
  }  
}
```

```
export default class FestaInfantil implements FabricaEvento {  
  criarAlimentacao(): Alimentacao { ...  
  }  
  criarMusica(): Musica { ...  
  }  
}
```

```
export default interface Musica {  
  obterTempoDuracao(): string  
  obterNome(): string  
  obterAutor(): string  
}
```

```
export default interface Alimentacao {  
  obterDescricao(): string  
}
```

Fábrica abstrata



Fábrica abstrata

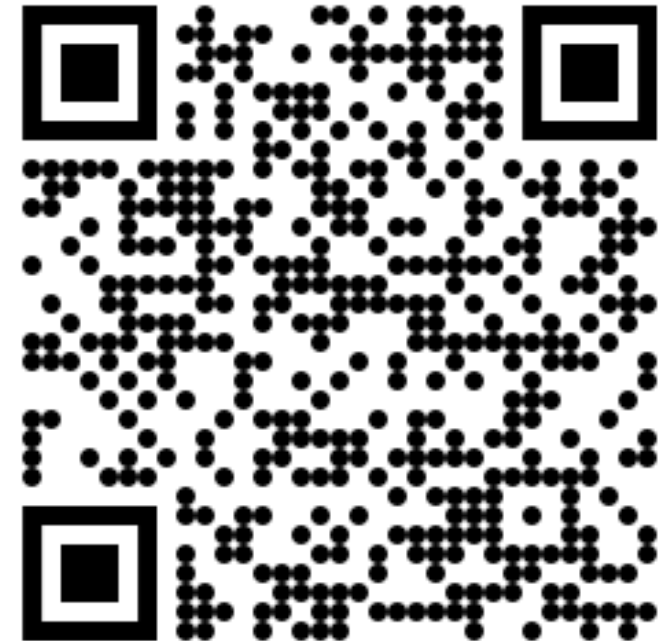


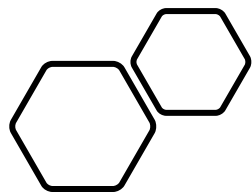
<https://github.com/gerson-pn>

[/abstract-factory-example](#)



[/abstract-factory-typeScript](#)





TypeScript

